## Department of Computing

Course Code: **CS332** — Class: **BSCS-13**

# Lab 09

## Indexing, Importing and Searching Data in Apache Solr

**Instructor:** Dr. Shah Khalid



| | |
|---|---|
| **Name** | Abrar Butt |
| **Qalam ID** | 460297 |
| **Section** | C |

## Contents

# Introduction

## Indexing Data

Indexing enables users to locate information in a document. It collects, parses, and stores documents to increase the speed and performance of a search query. When data is indexed in Solr, it is broken into tokens, analyzed, and stored in an inverted index structure that enables extremely fast retrieval.

## Importing Data

A Solr index can accept data from many different sources, including:

- **XML files** – sent via HTTP requests to the Solr server
- **CSV files** – comma-separated values ingested using the post tool
- **JSON files** – JavaScript Object Notation documents posted directly
- **Binary/Office files** – via the Solr Cell framework (Apache Tika)
- **Database tables** – via the DataImportHandler
- **Java API** – custom applications using Solr's Java Client API

## Searching

Solr offers a rich, flexible set of features for search. When a user runs a search, the query is processed by a **request handler** – a Solr plug-in defining the logic for request processing. Solr supports a variety of request handlers for search queries, index replication, and more. Search queries can be filtered, sorted, faceted, highlighted, and paginated using Solr's query syntax.

# Objectives

To index, import, and perform searching on sample data using Apache Solr in SolrCloud distributed mode.

# Tools / Software Requirement

- Ubuntu 24.04.1 LTS
- OpenJDK 21.0.10
- Apache Solr 8.11.4 (SolrCloud mode)
- Collection: `labdata` (2 shards, 2 replicas, 2 nodes)

# Lab Prerequisite

- Apache Solr installed and configured (completed in Lab 08)
- SolrCloud cluster with shards created (completed in Lab 08)

# Lab Task – Index, Import and Search Sample Data

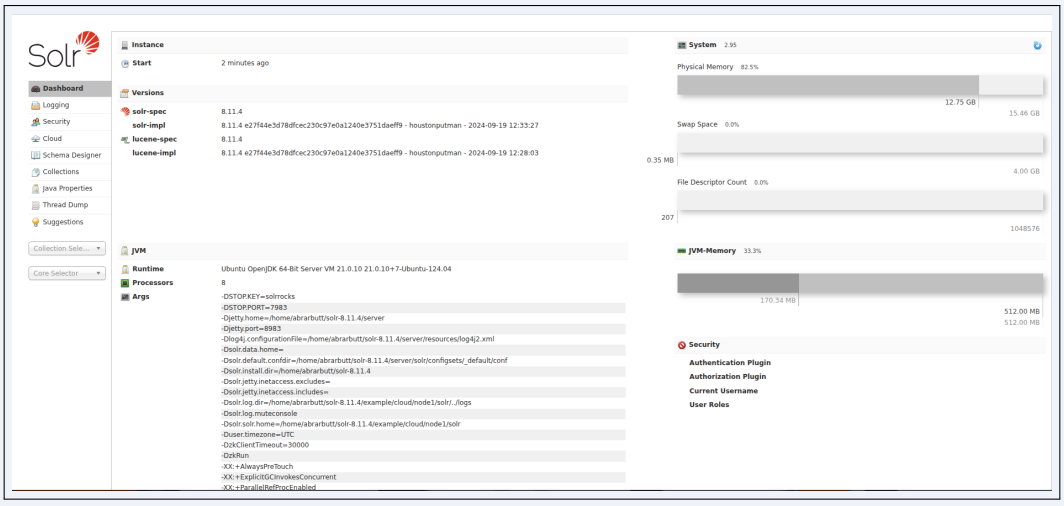
## Step 1 – Start SolrCloud Cluster

The SolrCloud cluster was started using the interactive `-e cloud` wizard with the following configuration:

```
cd ~/solr-8.11.4
bin/solr start -e cloud
```

| Parameter | Value |
|---|---|
| **Number of nodes** | 2 |
| **Node 1 port** | 8983 |
| **Node 2 port** | 7574 |
| **Collection name** | labdata |
| **Shards** | 2 (shard1, shard2) |
| **Replicas/shard** | 2 |
| **ZooKeeper port** | 9983 |

After the wizard completed, the Solr Admin UI was accessed at `http://localhost:8983/solr` confirming SolrCloud mode with Cloud, Collections, and Schema Designer visible in the left panel.




*Solr Admin Dashboard in SolrCloud mode – labdata collection active, Solr 8.11.4, JVM 512MB, 8 processors*

## Step 2 – Index & Import Sample Data

Solr ships with built-in example documents covering XML, CSV, and JSON formats. All were indexed into the `labdata` collection.

### Step 2a: Index XML example documents (multiple files):

```
bin/post -c labdata example/exampledocs/*.xml
```

This indexed all XML files including products, hardware, software, books, and company data.

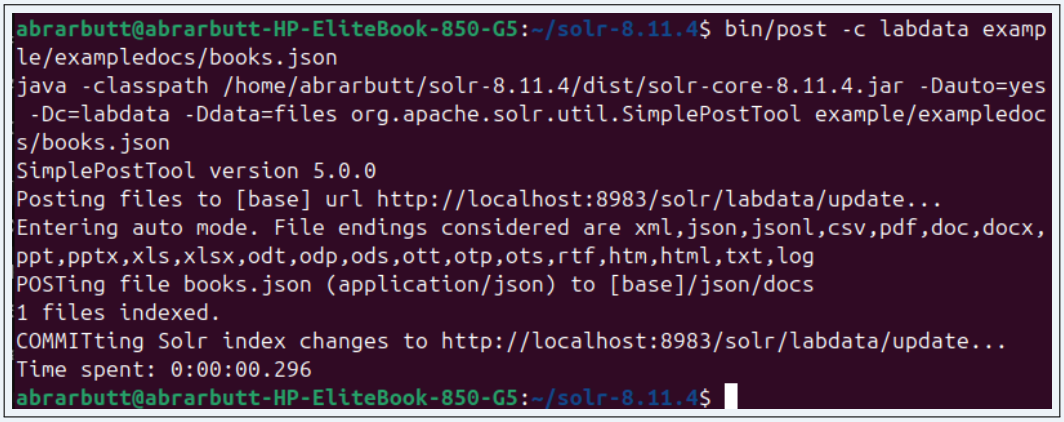
### Step 2b: Index CSV data (books):

```
bin/post -c labdata example/exampledocs/books.csv
```

### Step 2c: Index JSON data (books):

```
bin/post -c labdata example/exampledocs/books.json
```

All three indexing operations completed successfully with `COMMITting Solr index changes` confirmation. The total documents indexed across all formats came to **46 documents**.



```
abrarbutt@abrarbutt-HP-EliteBook-850-G5:~/solr-8.11.4$ bin/post -c labdata examp
le/exampledocs/books.json
java -classpath /home/abrarbutt/solr-8.11.4/dist/solr-core-8.11.4.jar -Dauto=yes
 -Dc=labdata -Ddata=files org.apache.solr.util.SimplePostTool example/exampledoc
s/books.json
SimplePostTool version 5.0.0
Posting files to [base] url http://localhost:8983/solr/labdata/update...
Entering auto mode. File endings considered are xml,json,jsonl,csv,pdf,doc,docx,
ppt,pptx,xls,xlsx,odt,odp,ods,ott,otp,ots,rtf,htm,html,txt,log
POSTing file books.json (application/json) to [base]/json/docs
1 files indexed.
COMMITting Solr index changes to http://localhost:8983/solr/labdata/update...
Time spent: 0:00:00.296
abrarbutt@abrarbutt-HP-EliteBook-850-G5:~/solr-8.11.4$
```

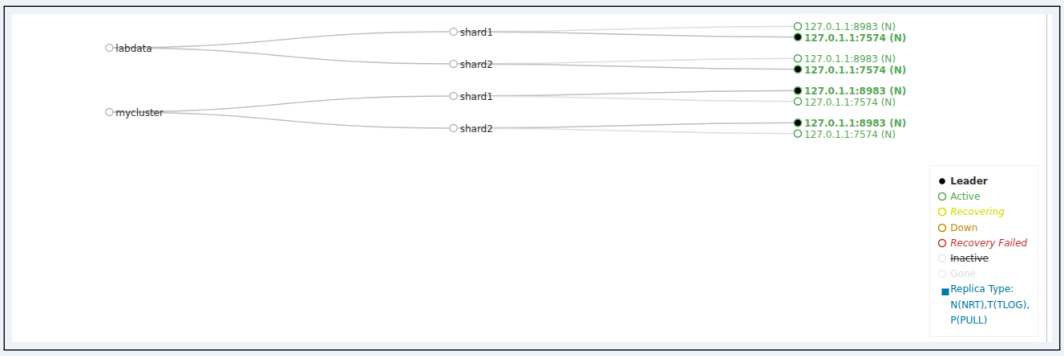
*Terminal output – books.json indexed into labdata collection (1 file, committed in 0.296s)*

**Total 46 documents indexed successfully into labdata across XML, CSV, and JSON formats.**

## Step 3 – Cluster Graph (Cloud View)

The SolrCloud graph was verified in the Admin UI under **Cloud** → **Graph**. It showed both collections (`labdata` and `mycluster` from Lab 08) with their shards and replicas distributed across the two nodes.

- **labdata**: shard1 & shard2, each replicated on ports 8983 and 7574
- **mycluster**: shard1 & shard2, each replicated on ports 8983 and 7574
- Leaders shown with filled circle (•), active replicas with green circle




*SolrCloud Graph – labdata and mycluster collections, each with 2 shards × 2 replicas across nodes 8983 & 7574*

## Step 4 – Searching Data

Solr was queried using the browser URL interface. All queries were sent to the `labdata` collection. The general query format is:

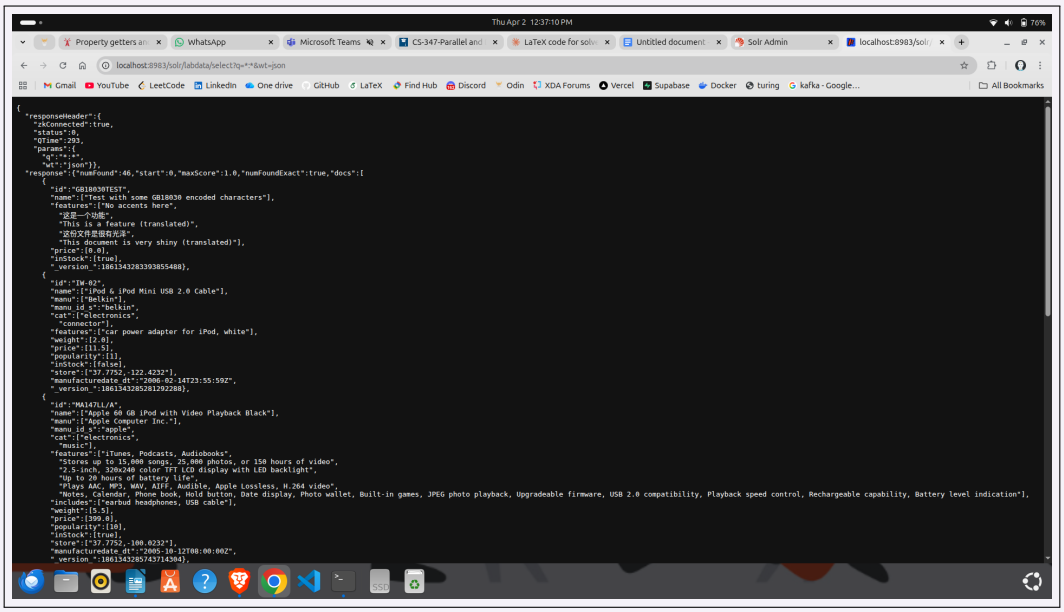
```
http://localhost:8983/solr/labdata/select?q=<QUERY>&wt=json
```

### Query 1: Search ALL Documents (q=*:*)

```
http://localhost:8983/solr/labdata/select?q=*:*&wt=json
```

#### Result – Query 1: All Documents

**numFound: 46** – All 46 indexed documents returned. Documents include electronics, books, companies, memory modules, cameras, and more. Sample records: GB18030TEST, IW-02 (iPod USB Cable), MA147LL/A (Apple iPod 60GB), A-DATA Technology, ASUS Computer, Belkin, Corsair XMS 2GB RAM, etc.




*Query q=*:* – All 46 documents returned from labdata collection distributed across both shards*

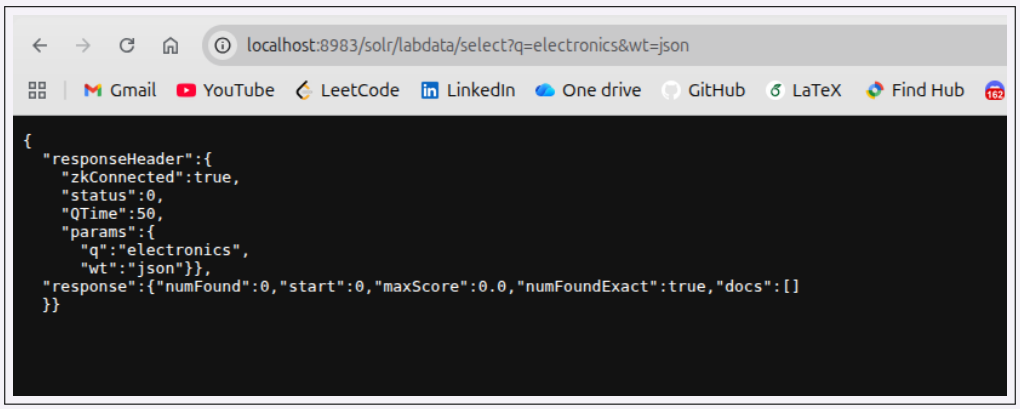
## Step 4 – Searching Data (Continued)

**Query 2: Search by keyword: electronics** (`q=electronics`)

```
http://localhost:8983/solr/labdata/select?q=electronics&wt=
    json
```

### Result – Query 2: Electronics

**numFound: 0** – The keyword `electronics` returned no results when searched as a full-text term across default fields. This is because the default search field in the `_default` configset does not include the `cat` field. To search by category, a field-specific query (`cat:electronics`) is needed.



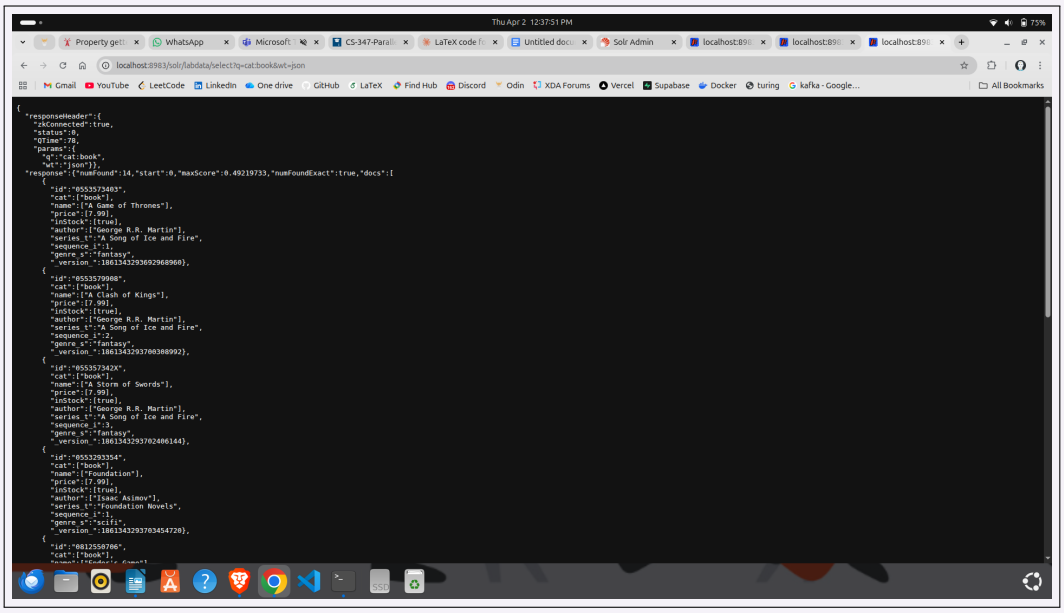

*Query q=electronics – numFound:0 (default field search; category field requires explicit field query)*

**Query 3: Search by category: books** (`q=cat:book`)

```
http://localhost:8983/solr/labdata/select?q=cat:book&wt=json
```

**Result – Query 3: cat:book**

**numFound: 14** – 14 book documents found. Results include: A Game of Thrones (George R.R. Martin), A Clash of Kings, A Storm of Swords, Foundation (Isaac Asimov), Ender's Game, and others from the CSV and JSON book datasets.




*Query q=cat:book – 14 book documents returned including Game of Thrones series, Foundation, and more*

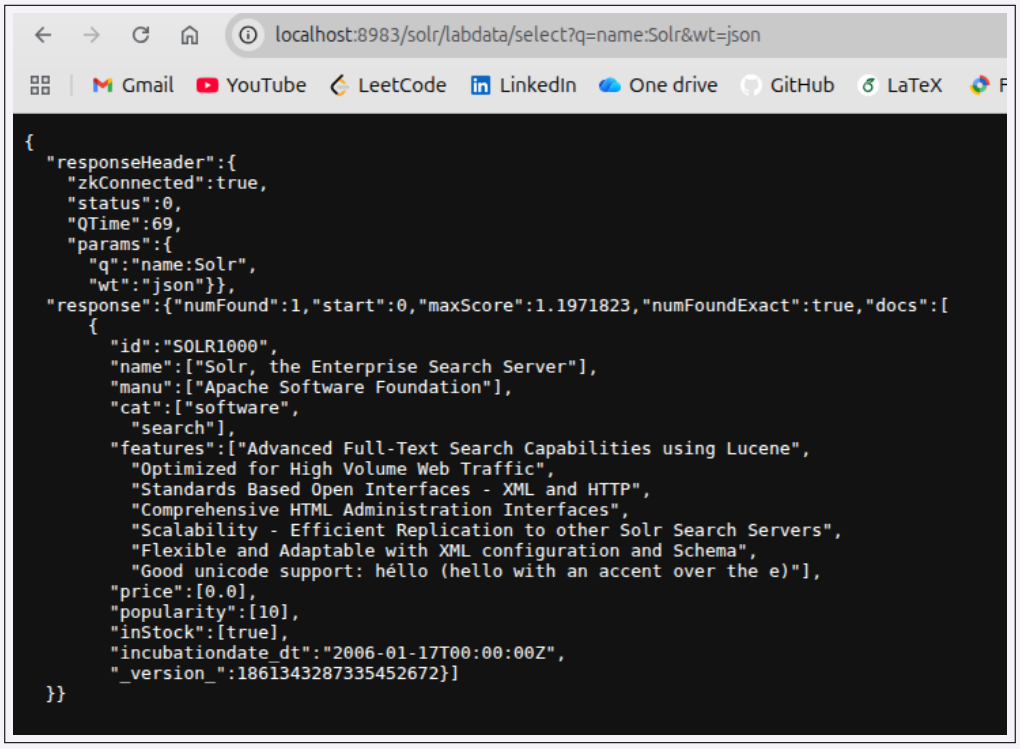
## Step 4 – Searching Data (Continued)

**Query 4: Search by name field: Solr** (`q=name:Solr`)

```
http://localhost:8983/solr/labdata/select?q=name:Solr&wt=json
```

### Result – Query 4: name:Solr

**numFound: 1** – One document found: `SOLR1000` – "Solr, the Enterprise Search Server" by Apache Software Foundation. Category: software, search. Features include Advanced Full-Text Search using Lucene, High Volume Web Traffic optimization, Standards Based XML and HTTP interfaces, and Scalability for replication. Price: $0.00, inStock: true.





```
{
  "responseHeader":{
    "zkConnected":true,
    "status":0,
    "QTime":69,
    "params":{
      "q":"name:Solr",
      "wt":"json"}},
  "response":{"numFound":1,"start":0,"maxScore":1.1971823,"numFoundExact":true,"docs":[
      {
        "id":"SOLR1000",
        "name":["Solr, the Enterprise Search Server"],
        "manu":["Apache Software Foundation"],
        "cat":["software",
          "search"],
        "features":["Advanced Full-Text Search Capabilities using Lucene",
          "Optimized for High Volume Web Traffic",
          "Standards Based Open Interfaces - XML and HTTP",
          "Comprehensive HTML Administration Interfaces",
          "Scalability - Efficient Replication to other Solr Search Servers",
          "Flexible and Adaptable with XML configuration and Schema",
          "Good unicode support: héllo (hello with an accent over the e)"],
        "price":[0.0],
        "popularity":[10],
        "inStock":[true],
        "incubationdate_dt":"2006-01-17T00:00:00Z",
        "_version_":1861343287335452672}]
  }}
```


*Query q=name:Solr – 1 document found: SOLR1000 "Solr, the Enterprise Search Server" with full feature list*

## Step 5 – Stop SolrCloud Cluster

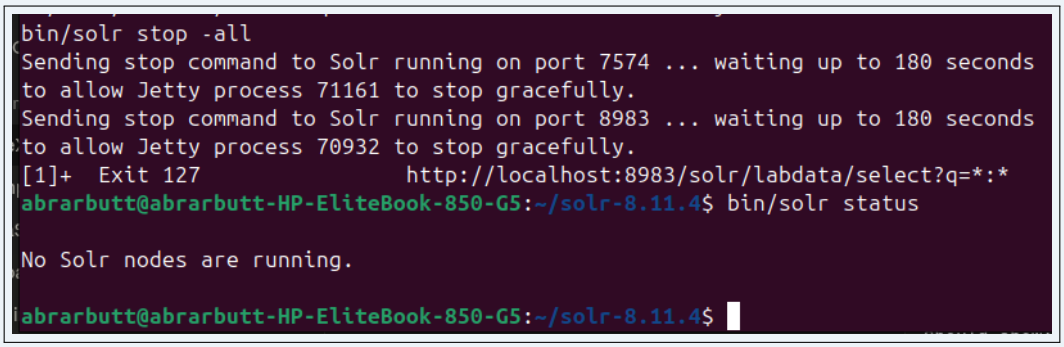
After completing all indexing and searching tasks, both SolrCloud nodes were stopped cleanly:

```
bin/solr stop -all
bin/solr status
```

Both Jetty processes were gracefully terminated:

- Port **7574** – Jetty process 71161 stopped
- Port **8983** – Jetty process 70932 stopped

Final status confirmed: *"No Solr nodes are running."*




*Both SolrCloud nodes stopped cleanly on ports 7574 and 8983 – No Solr nodes are running*

**All tasks completed. Cluster stopped cleanly.**

## Search Results Summary

| Query # | Query String | numFound | Description |
|---|---|---|---|
| **1** | `q=*:*` | **46** | All indexed documents returned |
| **2** | `q=electronics` | **0** | Full-text search on default field; use `cat:electronics` for field search |
| **3** | `q=cat:book` | **14** | Books indexed from CSV/JSON files |
| **4** | `q=name:Solr` | **1** | "Solr, the Enterprise Search Server" (SOLR1000) |

## Summary

| Item | Details |
|---|---|
| **OS** | Ubuntu 24.04.1 LTS |
| **Java Version** | OpenJDK 21.0.10 |
| **Solr Version** | Apache Solr 8.11.4 |
| **Collection** | `labdata` (2 shards × 2 replicas) |
| **Data Imported** | XML files, CSV (books), JSON (books) |
| **Total Docs Indexed** | 46 documents |
| **Searches Performed** | 4 queries (`*:*`, `electronics`, `cat:book`, `name:Solr`) |
| **Key Finding** | Field-specific queries (`cat:book`, `name:Solr`) work better than plain keyword searches in default configset |
| **Final State** | All nodes stopped cleanly |